Rapport d'Évaluation Pratique : INF4067
Système de Vente de Véhicules en Ligne

Matricule	**Nom et Prénom**
22U2079	LEUDJEU WOUAPPI BEAUTREL HORSSEL
22W2171	DJIMELI TCHINDA ARIANE
22U2142	TOUKAP NGANSOP RAYAN LEDOUX

Janvier 2026

Table des matières

1 Architecture du Système **2**
1.1 Architecture Technique 2

2 Solutions Conceptuelles par Besoin **4**
2.1 Besoin 1 : Création des objets du domaine 4
2.2 Besoin 2 : Construction des liasses de documents nécessaires 4
2.3 Besoin 3 : Création des commandes 5
2.4 Besoin 4 : Création de la liasse vierge (Singleton) 6
2.5 Besoin 5 : Gérer des documents PDF (Adapter) 6
2.6 Besoin 6 : Formulaires HTML ou Widgets (Bridge) 7
2.7 Besoin 7 : Représenter les sociétés clientes 7
2.8 Besoin 8 : Afficher les véhicules du catalogue 8
2.9 Besoin 9 : Retrouver sequentiellement les véhicules du catalogue 9
2.10 Besoin 10 : Calculer le montant d'une commande 11
2.11 Besoin 11 : Solder les véhicules en stock (Command) 11

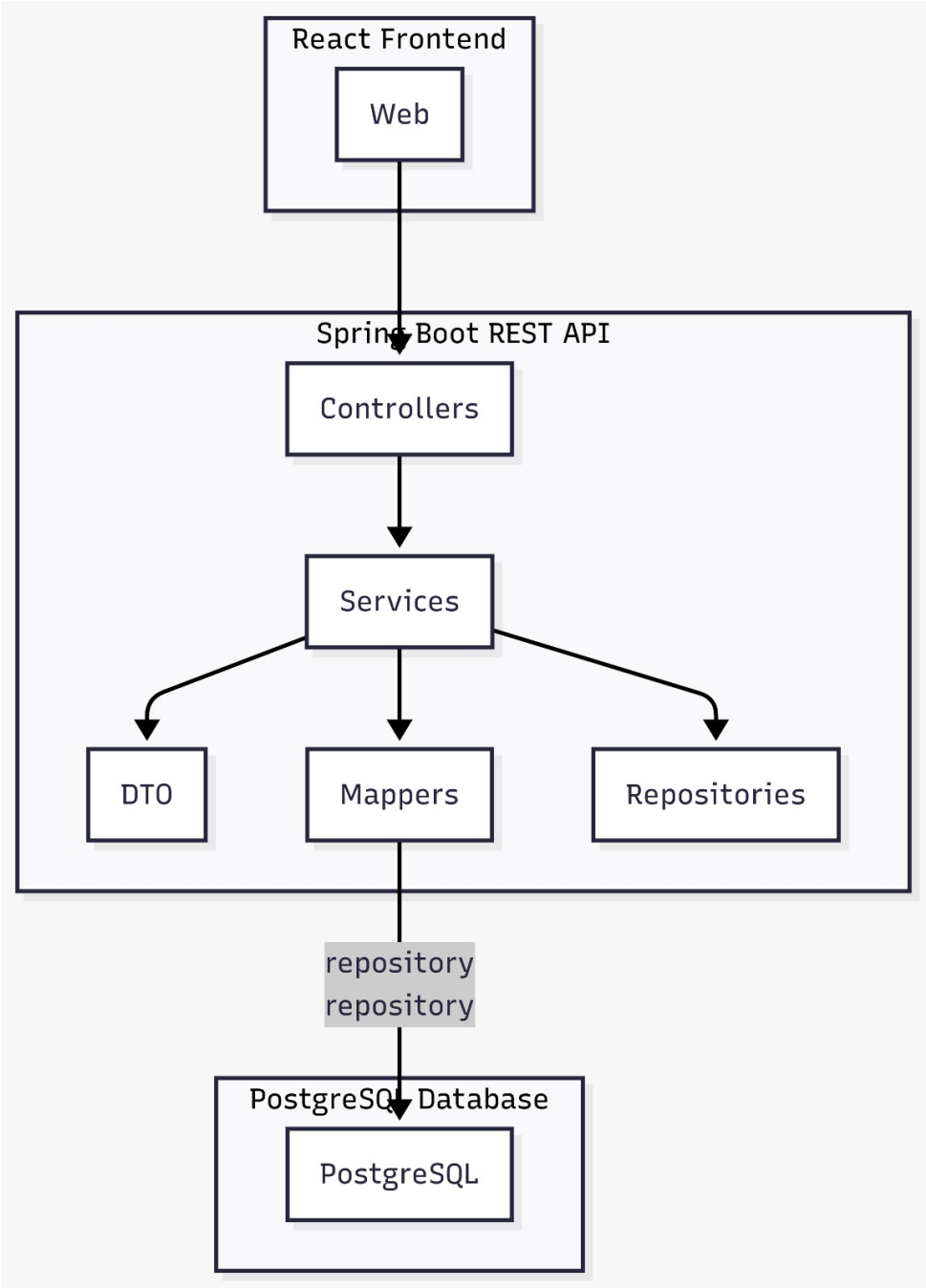
Chapitre 1

Architecture du Système

1.1 Architecture Technique

Le système repose sur une architecture moderne de type **Client-Serveur** :

- **Back-end :** Java avec le framework Spring Boot 4.0.0
- **Base de données :** Relationnelle(PostgreSQL) (JPA/Hibernate) pour la persistance des véhicules et commandes.
- **Front-end :** Application web (ReactJS).
- **API :** REST pour la communication entre les composants.

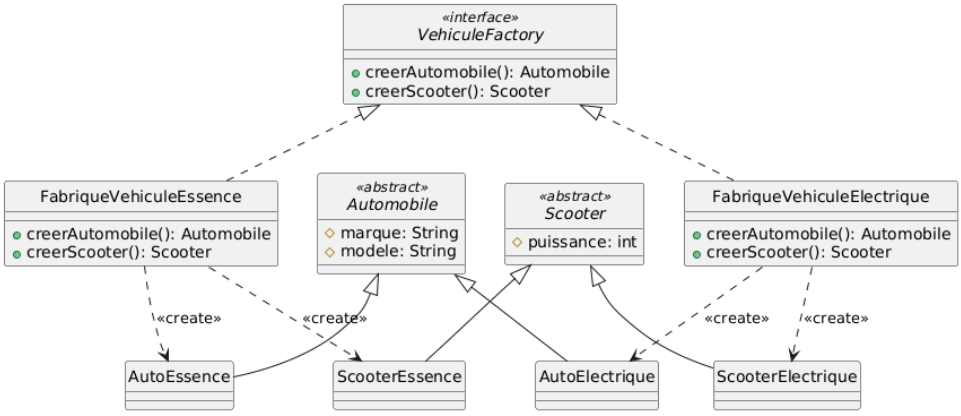

React Frontend
Web
Spring Boot REST API
Controllers
Services
DTO
Mappers
Repositories
repository
repository
PostgreSQL Database
PostgreSQL

Chapitre 2

Solutions Conceptuelles par Besoin

2.1 Besoin 1 : Création des objets du domaine

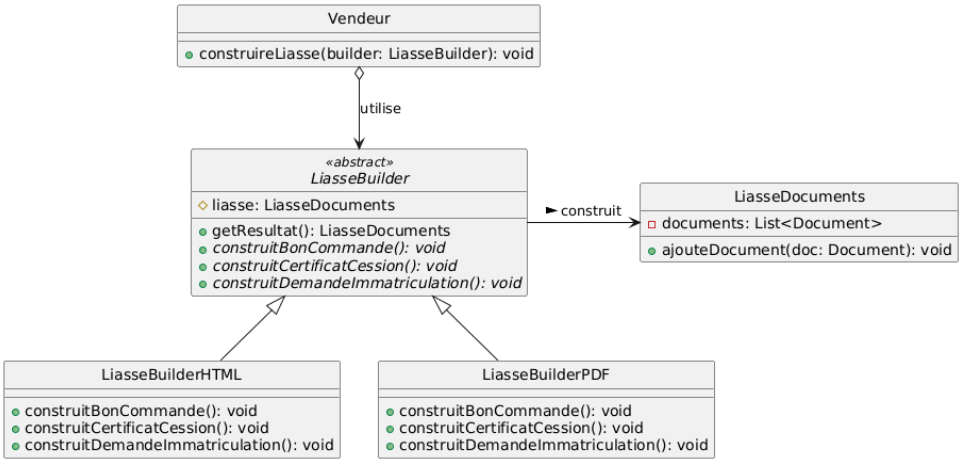
Explication du modèle : Le pattern Abstract Factory fournit une interface pour créer des familles d'objets liés (Automobiles et Scooters) sans spécifier leurs classes concrètes. Il garantit que les produits d'une même famille (ex : tout électrique) sont utilisés ensemble. VehiculeFactory (Fabrique Abstraite) : Interface qui déclare les méthodes de création pour chaque type de produit de la famille (creerAutomobile(), creerScooter()). FabriqueVehiculeEssence / Electrique (Fabriques Concrètes) : Implémentent les méthodes pour créer les objets d'une gamme spécifique (ex : la fabrique électrique crée des AutoElectrique et des ScooterElectrique). Automobile / Scooter (Produits Abstraits) : Définissent les caractéristiques communes (marque, modèle, puissance) pour chaque type de véhicule. AutoEssence / ScooterElectrique / etc. (Produits Concrets) : Les objets finaux créés par les fabriques respectives, regroupés par thématique technique.



2.2 Besoin 2 : Construction des liasses de documents nécessaires

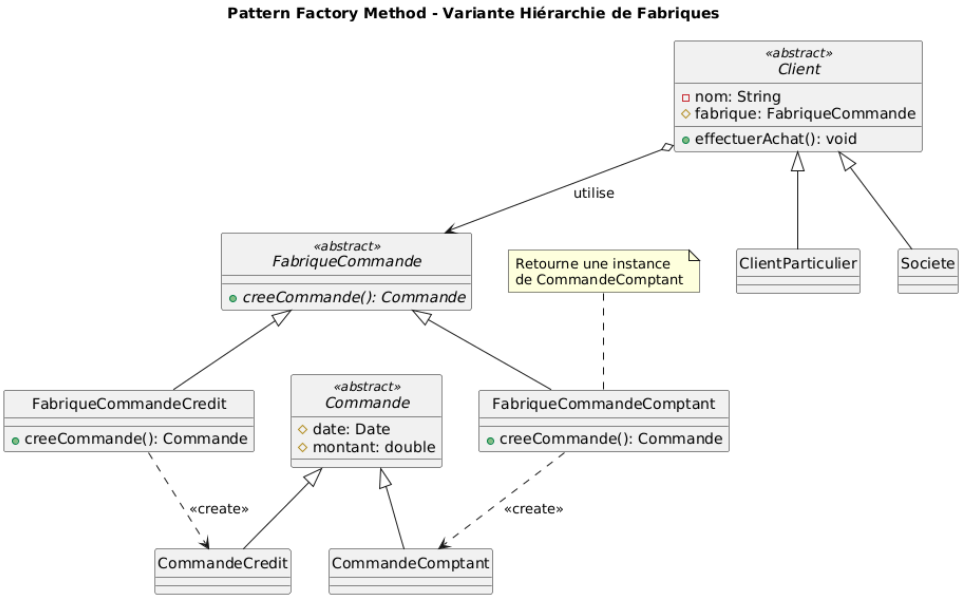
Explication du modèle : Le pattern **Builder** permet de séparer la logique de construction d'un objet complexe (la liasse de documents) de sa représentation finale (HTML ou PDF). Cela permet de créer différentes versions d'un même objet avec le même processus de construction. Vendeur (Directeur) : Pilote la construction en appelant les étapes du builder dans l'ordre via construireLiasse(). Il ne sait pas si la liasse sera en

PDF ou HTML. LiasseBuilder (Monteur Abstrait) : Définit les méthodes d'étape pour construire chaque document (Bon de commande, Certificat, etc.) et la méthode pour récupérer le résultat. LiasseBuilderHTML / PDF (Monteurs Concrets) : Implémentent la création spécifique des documents dans le format souhaité. LiasseDocuments (Produit) : Représente l'objet complexe final qui contient la liste des documents générés.



2.3 Besoin 3 : Création des commandes

Explication du modèle : Le pattern **Factory Method** permet de découpler la création des commandes de leur utilisation. Client (Créateur Abstrait) : Définit la signature de la "méthode fabrique" creeCommande(). Il contient également la logique métier générale dans nouvelleCommande() qui utilise le produit créé sans connaître sa classe concrète. ClientParticulier / Societe (Créateurs Concrets) : Implémentent la méthode creeCommande(). Par exemple, ClientParticulier pourrait instancier une CommandeComptant tandis que Societe instancierait une CommandeCredit. Commande (Produit Abstrait) : Définit l'interface ou la classe de base pour tous les types de commandes (date, montant). CommandeComptant / CommandeCredit (Produits Concrets) : Représentent les objets réels créés par les fabriques respectives.

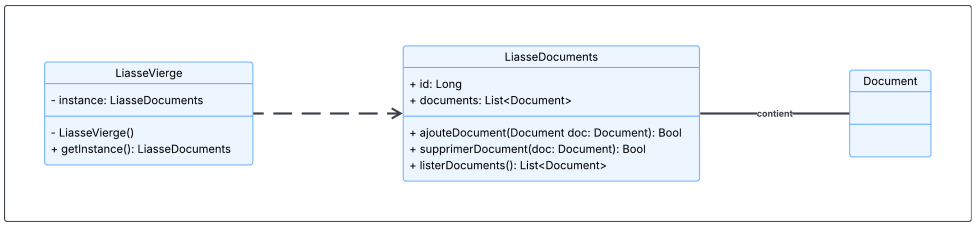


2.4 Besoin 4 : Création de la liasse vierge (Singleton)

Problématique : Garantir qu'il n'existe qu'un seul gestionnaire de documents pour créer des liasses vierges afin d'éviter les doublons mémoire.

Solution : Le patron **Singleton** restreint l'instanciation de la classe 'LiasseVierge' à un seul objet partagé par toute l'application.

Explication du modèle : L'implémentation repose sur une classe LiasseVierge possédant une instance statique privée d'elle-même. Le constructeur est rendu private pour empêcher toute instanciation externe via le mot-clé new. L'accès à l'unique instance se fait via une méthode statique getInstance(). Pour garantir la sécurité dans un environnement multi-thread (comme Spring Boot), nous utilisons le mécanisme du "Lazy Initialization" avec un bloc synchronized. Cela assure que même si plusieurs clients demandent une liasse simultanément, un seul objet sera créé et partagé, optimisant ainsi la consommation mémoire.

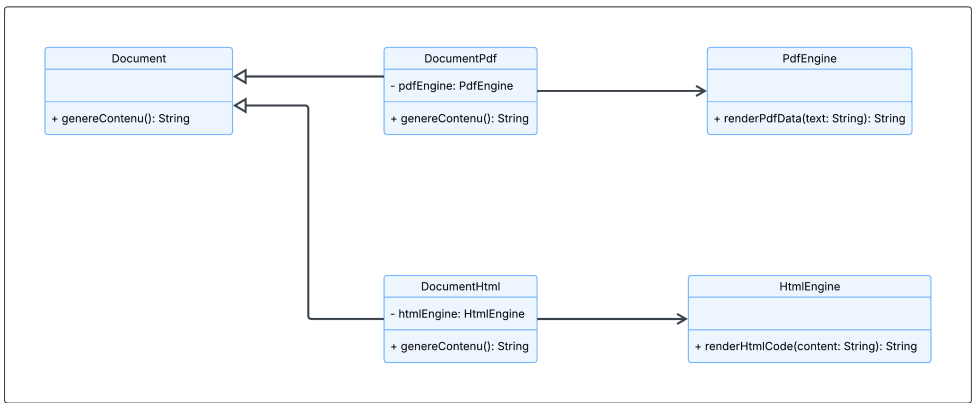


2.5 Besoin 5 : Gérer des documents PDF (Adapter)

Problématique : Intégrer un moteur de rendu PDF tiers dont l'interface n'est pas compatible avec notre interface interne 'Document'.

Solution : Le patron **Adapter** permet de convertir l'interface du moteur PDF en une interface attendue par notre système, agissant comme un traducteur.

Explication du modèle : Le système définit une abstraction Document avec une méthode genereContenu(). L'adaptateur DocumentPdf implémente notre abstraction Document et contient une référence vers le ComposantPdf. Lorsqu'on appelle genereconten() sur l'adaptateur, celui-ci traduit l'appel en instructions compréhensibles par le moteur PDF. Il en est de même pour le composantHtml. Ce modèle permet d'intégrer des bibliothèques tierces sans modifier le code source de notre application.

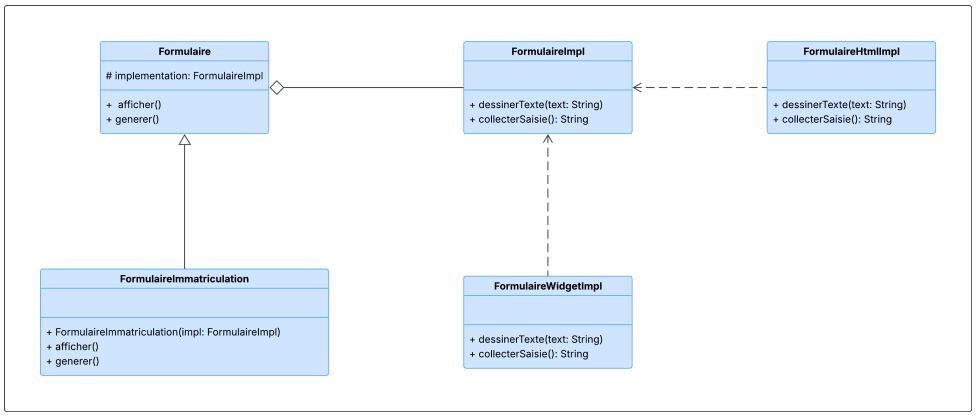


2.6 Besoin 6 : Formulaires HTML ou Widgets (Bridge)

Problématique : Séparer l'abstraction du formulaire (le contenu) de son implémentation (le rendu visuel HTML ou Widget) pour qu'ils puissent varier indépendamment.

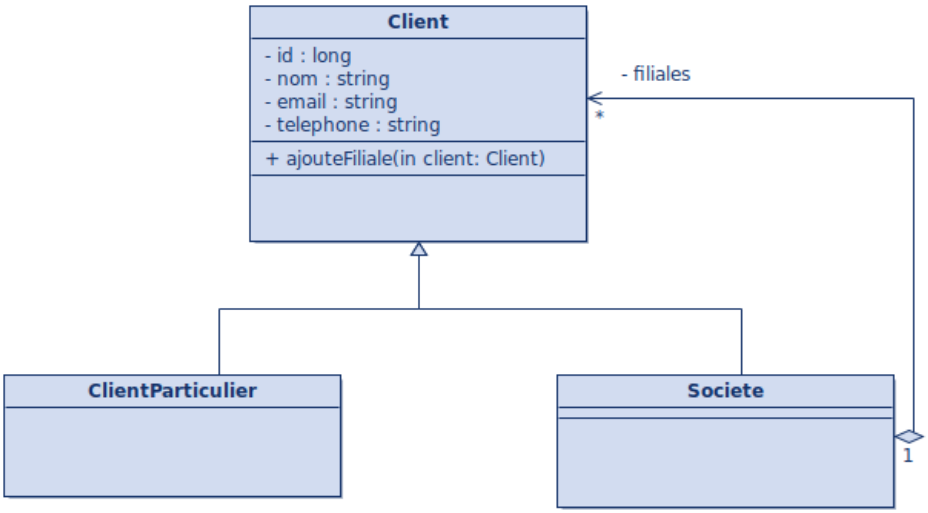
Solution : Le patron **Bridge** découple l'interface 'Formulaire' de l'implémentation 'Rendu', permettant d'ajouter de nouveaux types de rendus sans modifier les formulaires existants.

Explication du modèle : Ce pattern sépare l'abstraction Formulaire de son implémentation de rendu FormulaireImpl. Nous avons deux hiérarchies indépendantes : d'un côté les types de formulaires (ex : FormulaireImmatriculation) et de l'autre les formats de sortie (ex : FormulaireHtmlImpl, FormulaireWidgetImpl). L'abstraction détient une référence vers l'implémentation. Cette structure permet de créer un nouveau type de formulaire sans se soucier du rendu, ou d'ajouter un nouveau format d'affichage sans modifier les formulaires existants.



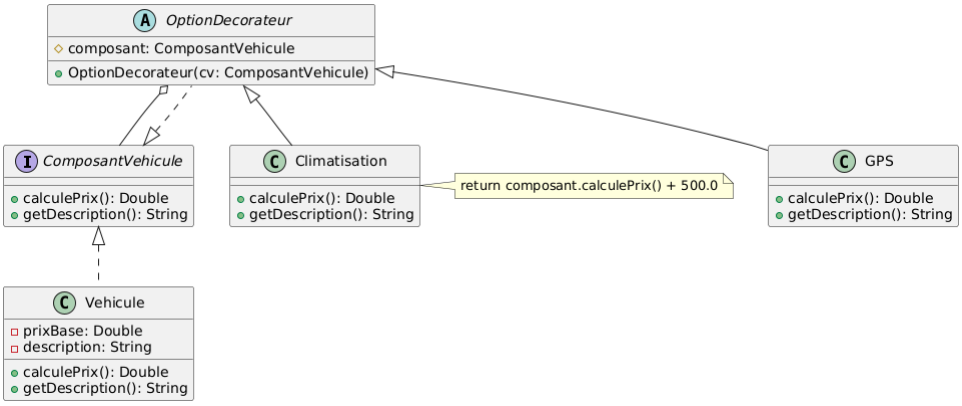
2.7 Besoin 7 : Représenter les sociétés clientes

Explication du modèle : Le pattern **Composite** permet de traiter de la même manière un client individuel et une structure complexe de sociétés. Client (Composant) : Définit l'interface commune pour tous les types de clients, qu'ils soient simples ou composés. ClientParticulier (Feuille) : Représente les clients finaux. Ils héritent de Client mais ne possèdent pas de filiales. Societe (Composite) : Représente un regroupement de clients. Elle stocke une liste de filiales (relation vers Client). Comme elle considère ses filiales comme des types Client, elle peut contenir aussi bien des particuliers que d'autres sociétés, permettant ainsi une récursivité infinie.

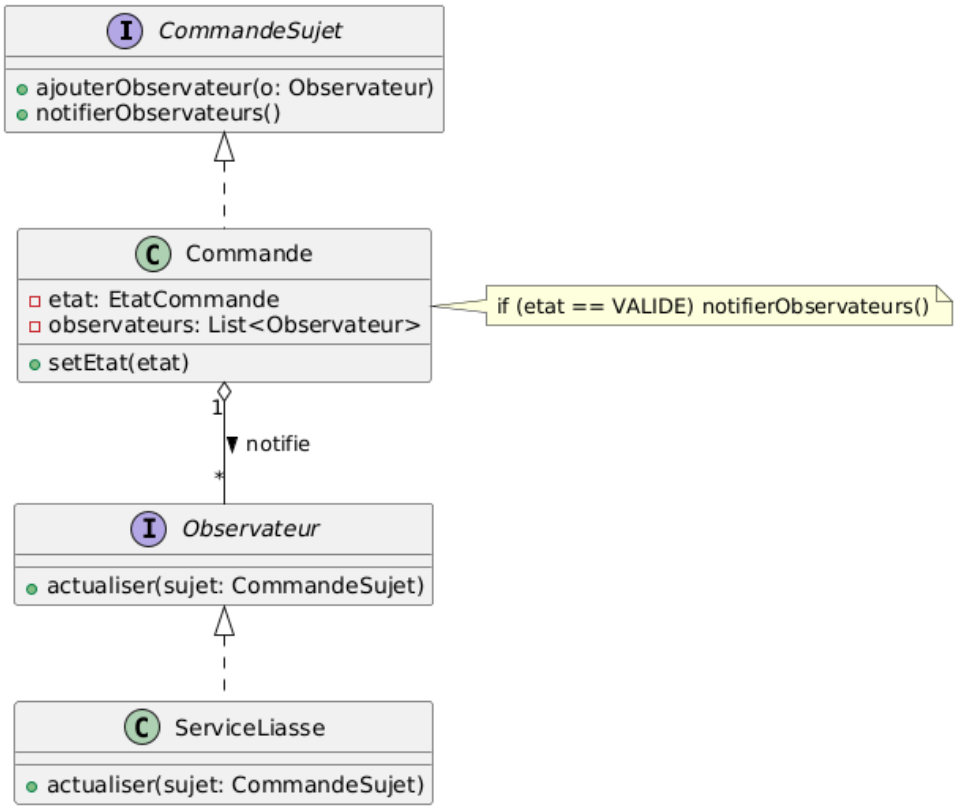


2.8 Besoin 8 : Afficher les véhicules du catalogue

Explication du modèle : Le pattern **Decorator** permet d'ajouter dynamiquement le prix et la description des options à un véhicule sans modifier la classe Vehicule. ComposantVehicule (Interface/Abstract) : Définit calculePrix() et getDescription(). Vehicule (Classe Concrète) : L'entité de base qui implémente ces méthodes. OptionDecorateur (Décorateur) : Contient une instance de ComposantVehicule. Logique : Lorsqu'on appelle calculePrix() sur le décorateur, il fait : véhiculeInterne.calculePrix() + prixOption.

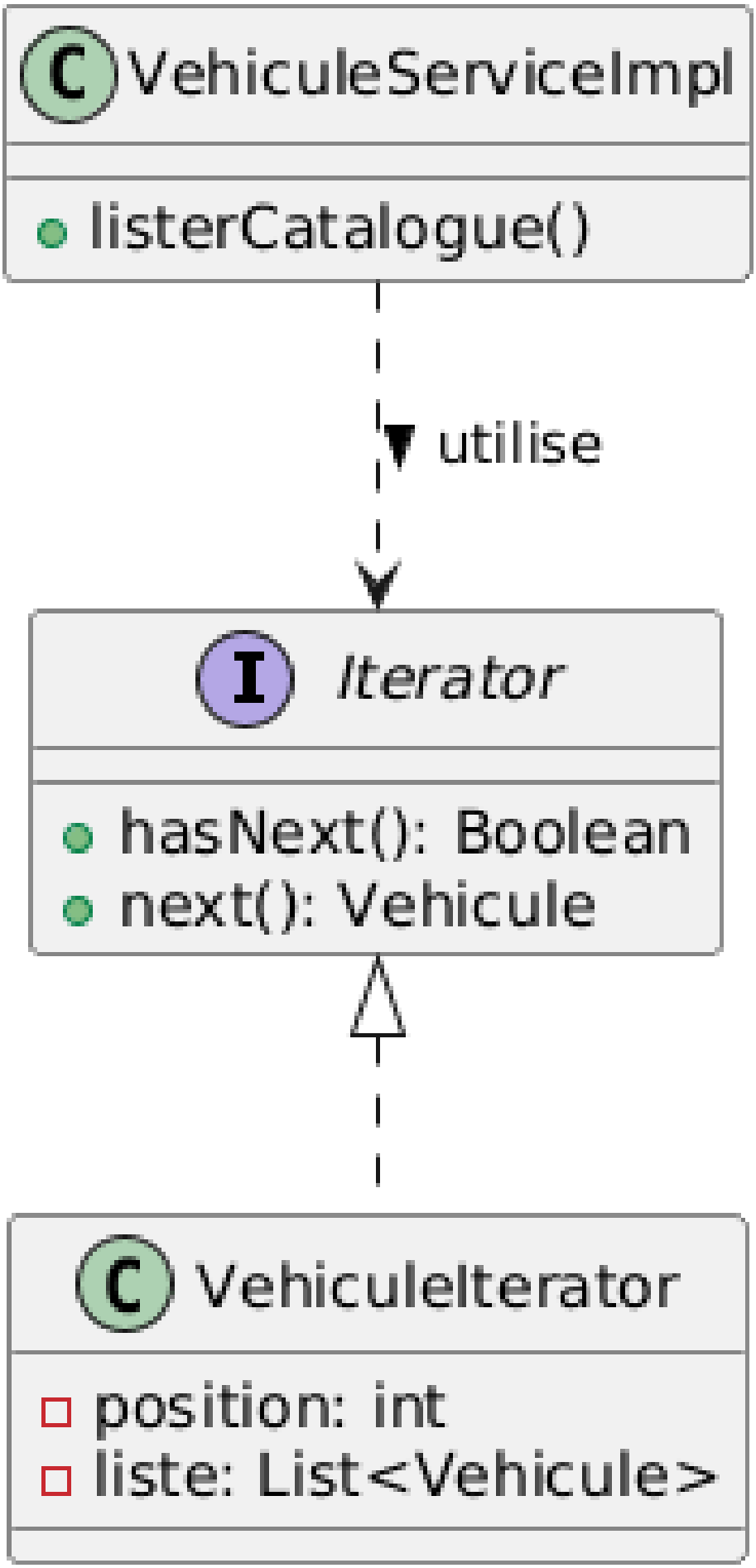


Explication du modèle : Le pattern **Observer** permet de découpler la gestion de la commande et la génération des documents. CommandeSujet (Interface) : Définit les méthodes pour gérer les observateurs (ajouter, notifier). Commande (Sujet Concret) : Stocke l'état. Quand l'état passe à VALIDE, elle appelle notifierObservateurs(). Observateur (Interface) : Définit la méthode actualiser(). ServiceLiasse (Observateur Concret) : Réagit à la notification en lançant la construction de la liasse.



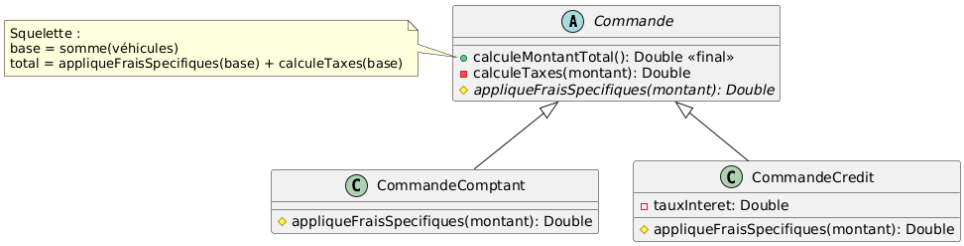
2.9 Besoin 9 : Retrouver sequentiellement les véhicules du catalogue

Explication du modèle : Le pattern **Iterator** abstrait la façon dont on parcourt la liste de véhicules en base de données. VehiculeIterator (List de Spring/Java) : La collection de véhicules. Iterator : L'objet qui possède hasNext() et next(). VehiculeServiceImpl : Utilise l'itérateur pour transformer chaque Vehicule en VehiculeResultDTO sans se soucier de si la collection est une ArrayList ou une LinkedList.


VehiculeServiceImpl
listerCatalogue()
utilise
Iterator
hasNext(): Boolean
next(): Vehicule
VehiculeIterator
position: int
liste: List<Vehicule>

2.10 Besoin 10 : Calculer le montant d'une commande

Explication du modèle : Le pattern **Template Method** définit les étapes fixes du calcul du prix tout en laissant les sous-classes gérer les frais spécifiques. Commande (Classe Abstraite) : Contient la méthode calculeMontantTotal() (le "squelette"). Elle appelle la méthode abstraite appliqueFraisSpecifiques(). CommandeComptant : Implémente la méthode spécifique (retourne le montant tel quel). CommandeCredit : Implémente la méthode spécifique (ajoute les intérêts via le tauxInteret).

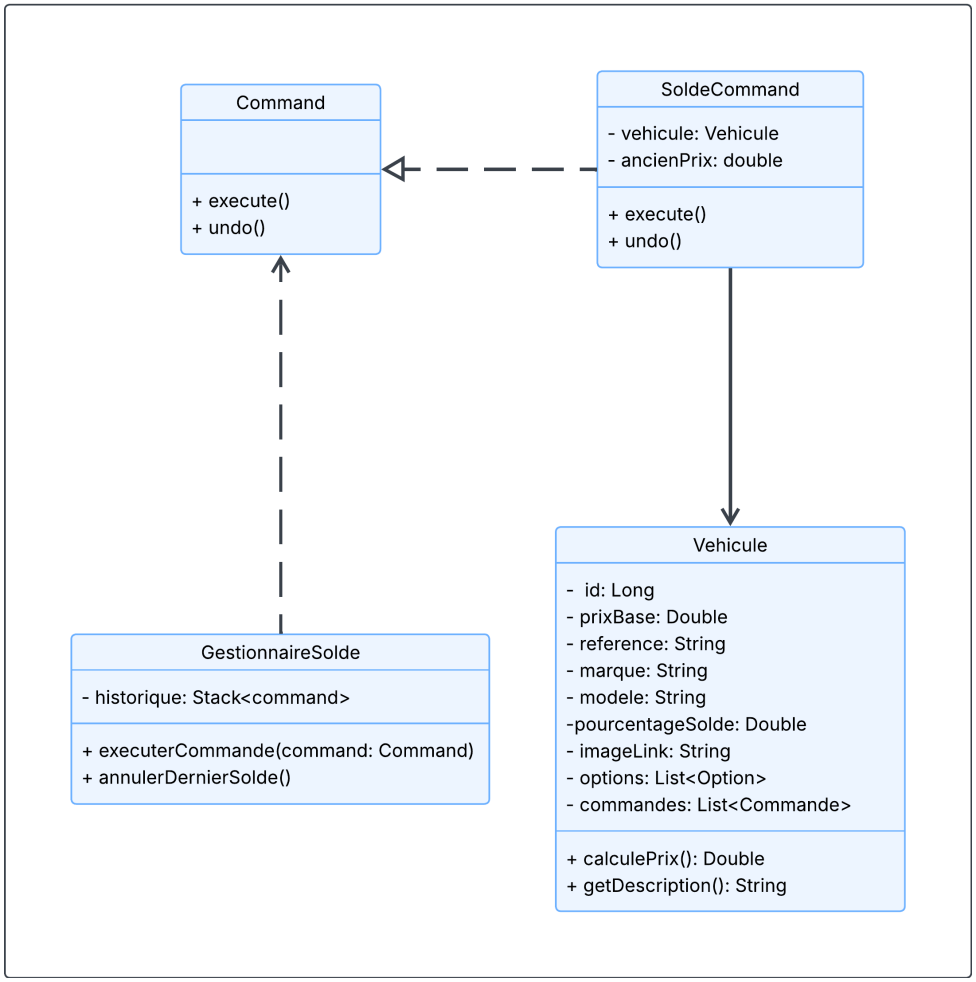


2.11 Besoin 11 : Solder les véhicules en stock (Command)

Problématique : Appliquer des remises sur le prix des véhicules en stock de manière réversible (possibilité d'annuler la solde).

Solution : Le patron **Command** encapsule la modification du 'prixBase' dans un objet 'SoldeCommand'. Cela permet de garder un historique des actions et d'implémenter la méthode 'undo()' pour restaurer le prix initial.

Explication : L'invocateur ('GestionnaireSolde') gère une pile de commandes. Chaque commande agit sur le récepteur ('Vehicule'). Cette approche respecte le principe OCP (Open-Closed Principle) car on peut ajouter de nouvelles opérations de masse sans modifier la classe 'Vehicule'.


Command
+ execute()
+ undo()
SoldeCommand
- vehicule: Vehicule
- ancienPrix: double
+ execute()
+ undo()
GestionnaireSolde
- historique: Stack<command>
+ executerCommande(command: Command)
+ annulerDernierSolde()
Vehicule
- id: Long
- prixBase: Double
- reference: String
- marque: String
- modele: String
-pourcentageSolde: Double
- imageLink: String
- options: List<Option>
- commandes: List<Commande>
+ calculePrix(): Double
+ getDescription(): String